MODULE 6 : Intro to AI & Mini Projects

Chapter — Introduction to AI, Machine Learning and Python

1. Learning Objectives

After completing this chapter, a learner should be able to:

- Explain Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL).
- Understand the relationship between AI, ML, and DL.
- Explain how Python is used in AI and ML.
- Understand the basic ML workflow.
- Identify features, labels, training data, and testing data.
- Understand supervised learning.
- Distinguish between regression and classification.
- Install and import scikit-learn.
- Train a basic machine-learning model.
- Make predictions using a trained model.
- Evaluate a model using suitable metrics.
- Understand why training and testing data must be separated.
- Build simple regression applications.
- Build simple classification applications.

- Perform basic text classification.
- Understand the basic idea behind sentiment analysis.
- Build a rule-based chatbot.
- Understand the difference between a rule-based chatbot and an ML/AI chatbot.

---

2. What is Artificial Intelligence?

Simple Definition

Artificial Intelligence (AI) is a broad field of computing concerned with building systems that can perform tasks that normally require capabilities associated with human intelligence.

Examples include:

- Understanding text
- Recognizing objects in images
- Translating languages
- Recommending products
- Detecting suspicious transactions
- Predicting demand
- Answering questions
- Planning and decision-making

AI is a broad field. It does **not** mean that every AI system learns from data.

For example, a simple rule-based system can be considered an AI application even though it does not train a machine-learning model.

Example

A customer-support system could contain rules:

```
IF customer says "refund"
    → show refund instructions

IF customer says "password"
    → show password-reset instructions
```

This system is automated and can perform a task associated with intelligent behavior, but it is not necessarily machine learning.

---

3. What is Machine Learning?

Simple Definition

Machine Learning (ML) is a subset of AI in which algorithms learn patterns from data and use those learned patterns to make predictions or decisions on new data.

Instead of explicitly programming every possible rule, we provide examples to a learning algorithm.

Traditional Programming

```
Rules + Data → Output
```

Example:

```
IF temperature > 38:
    print("High temperature")
```

The programmer explicitly defines the rule.

Machine Learning

```
Data + Expected Outputs → Model
```

The model learns a relationship from examples.

Later:

```
New Data → Trained Model → Prediction
```

---

4. Real-Life Example of Machine Learning

Suppose a company wants to predict whether a customer may cancel a subscription.

The company may have historical data:

Age	Monthly Bill	Months With Company	Support Tickets	Cancelled
23	499	3	5	Yes
35	799	24	1	No
28	599	8	4	Yes
42	999	36	0	No

The model can learn patterns from previous customers.

For a new customer:

```
Age = 30
Monthly Bill = 699
Months With Company = 10
Support Tickets = 3
```

the model can estimate:

```
Likely to cancel → Yes
```

This is a **classification problem** because the output belongs to a category.

---

5. What is Deep Learning?

Deep Learning (DL) is a subfield of machine learning that uses neural networks with multiple layers to learn increasingly complex representations from data.

Deep learning is particularly important in areas such as:

- Computer vision
- Speech recognition
- Natural language processing
- Generative AI
- Image generation
- Large language models

Examples of deep-learning applications include:

- Face recognition
- Speech-to-text
- Object detection
- Machine translation
- Large language models

---

6. Relationship Between AI, ML and DL

A useful way to remember the relationship is:

```
Artificial Intelligence
│
├── Rule-based AI
│
└── Machine Learning
    │
    ├── Traditional ML
    │
    └── Deep Learning
        │
        ├── Neural Networks
```

```
        ├── CNN
        ├── RNN
        └── Transformer-based models
```

This diagram is conceptual rather than a complete taxonomy.

Important Point

Not all AI is machine learning.

Not all machine learning is deep learning.

Deep learning is a type of machine learning.

---

7. AI vs ML vs DL

Technology	Basic Idea	Example
AI	Broad field of intelligent computer systems	Automated decision system
ML	Learns patterns from data	Spam detection
DL	ML using multi-layer neural networks	Image recognition

---

8. Why Python is Used in AI and ML

Python is widely used for AI and ML because it has:

- Simple and readable syntax
- Large ecosystem of libraries
- Strong data-processing capabilities
- Scientific-computing libraries
- Machine-learning libraries
- Visualization libraries
- Large developer community
- Easy integration with other technologies

Python itself does not provide "AI automatically".

Instead, Python acts as the programming language through which developers use libraries and frameworks.

The Python ecosystem includes libraries such as:

Library	Common Purpose
NumPy	Numerical computing
Pandas	Data manipulation
Matplotlib	Visualization
Seaborn	Statistical visualization
Scikit-learn	Traditional machine learning
TensorFlow	Deep learning
PyTorch	Deep learning
Transformers	Modern NLP/LLM workflows

Python's official documentation describes Python as a general-purpose programming language with extensive libraries and support for rapid application development.

---

9. What is scikit-learn?

scikit-learn is an open-source Python machine-learning library.

It provides tools for:

- Classification
- Regression
- Clustering
- Data preprocessing
- Feature extraction
- Model selection
- Model evaluation
- Pipelines

The current stable documentation describes scikit-learn as providing tools for predictive data analysis and supporting many machine-learning algorithms.

Official website:

scikit-learn Documentation

---

10. Installing scikit-learn

Open a terminal or command prompt:

```
pip install scikit-learn
```

For data-analysis projects, you will commonly also install:

```
pip install numpy pandas matplotlib
```

You can verify the installation:

```
import sklearn

print(sklearn.__version__)
```

---

11. Basic Machine Learning Terminology

Before writing ML programs, understand these terms.

Dataset

A collection of data used by a machine-learning system.

Example:

```
student data
customer data
```

```
transaction data
product data
```

Feature

A feature is an input variable used by the model.

Example:

```
Hours studied
Attendance
Previous marks
```

These can be features for predicting examination marks.

Label / Target

The value that the model is expected to predict.

Example:

```
Hours studied → Feature

Exam marks → Target
```

Sample

One row or individual observation in a dataset.

Example:

```
5 hours studied → 72 marks
```

is one sample.

---

12. Training Data

Training data is the data used to learn the model's parameters.

Example:

```
Hours    Marks
2        42
3        48
4        55
5        63
6        70
```

The model learns a relationship between hours and marks.

---

13. Testing Data

Testing data is separate data used to evaluate how the trained model performs on data it did not train on.

This is important because a model that performs well on training data may not perform well on unseen data.

---

14. Training vs Testing

A common workflow is:

```
Complete Dataset
       |
       v
Train/Test Split
   /           \
  v             v
Training      Testing
  |             |
  v             v
Train Model   Evaluate Model
```

Example:

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

Here:

- `X` = input features
- `y` = target
- `X_train` = training features
- `X_test` = testing features
- `y_train` = training targets
- `y_test` = testing targets

---

15. Supervised Learning

In supervised learning, the training dataset contains known target values.

Example:

```
Hours Studied → Marks
```

The model learns from examples where the correct output is already known.

Two important supervised-learning tasks are:

1. Regression
2. Classification

---

16. Regression

Regression is used when the target is a continuous numerical value.

Examples:

```
Predict house price
Predict sales revenue
Predict electricity consumption
Predict delivery time
Predict student marks
```

Example:

```
Input:
House size = 1500 sq ft

Output:
Predicted price = ₹75,00,000
```

The output is numerical.

Scikit-learn defines regression as predicting a continuous-valued attribute.

---

17. Linear Regression

Linear regression attempts to model the relationship between input variables and a continuous target using a linear function.

For one input feature:

```
y = b0 + b1x
```

Where:

- `x` = input feature
- `y` = predicted value
- `b0` = intercept

- `b1` = coefficient

For example:

```
Hours studied → Exam marks
```

A simple model may learn approximately:

```
Marks = 30 + 7 × Hours
```

This is a simplified example. Real-world data will rarely follow such a perfectly linear relationship.

---

18. Important Limitation of Linear Regression

Linear regression should not automatically be used for every prediction problem.

For example:

```
Predict customer churn
```

has an output such as:

```
Yes / No
```

That is a classification problem, not a standard regression problem.

Likewise, linear regression should not be assumed to provide reliable predictions simply because a relationship appears visually linear.

The dataset, assumptions, feature quality, and evaluation results matter.

---

19. Classification

Classification predicts a category or class.

Examples:

```
Spam / Not Spam
Fraud / Not Fraud
Positive / Negative
Approved / Rejected
Churn / No Churn
```

Example:

```
Email text
    ↓
Classification Model
    ↓
Spam
```

Scikit-learn describes classification as identifying which category an object belongs to.

---

20. Common Classification Algorithms

Examples include:

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine
- K-Nearest Neighbors
- Naive Bayes
- Gradient Boosting

For this chapter, we will concentrate on:

```
Linear Regression
Logistic Regression
Decision Tree
```

The important distinction is:

```
Linear Regression → Regression

Logistic Regression → Classification
```

Despite its name, logistic regression is commonly used for classification.

---

21. Binary Classification

Binary classification has two classes.

Examples:

```
Spam / Not Spam
Positive / Negative
Fraud / Genuine
Yes / No
```

Example:

```
Transaction
     ↓
Model
     ↓
Fraud / Not Fraud
```

---

22. Multiclass Classification

Multiclass classification has more than two classes.

Example:

```
Support Ticket
       ↓
Classification Model
       ↓
Billing
Technical
Account
Shipping
```

The model chooses one class from multiple possible classes.

---

23. The Basic ML Workflow

A practical machine-learning project generally follows a workflow such as:

```
1. Define the problem
        ↓
2. Collect data
        ↓
3. Understand the data
        ↓
4. Clean/preprocess data
        ↓
5. Select features
        ↓
6. Split training/testing data
        ↓
7. Select a model
        ↓
8. Train the model
        ↓
9. Evaluate the model
        ↓
10. Improve the model
        ↓
11. Test with new data
        ↓
12. Deploy/monitor if required
```

A model is only one part of an ML system.

---

24. The `fit()` and `predict()` Pattern

One of the most important patterns in scikit-learn is:

```
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

`fit()` trains the estimator on the training data.

`predict()` generates predictions for new input data.

Scikit-learn's estimator API is designed around this general fitting and prediction pattern.

---

25. Model Evaluation

A model should not be judged only by looking at a few predictions.

Different problems require different evaluation metrics.

Regression Metrics

Common metrics include:

- MAE — Mean Absolute Error
- MSE — Mean Squared Error
- RMSE — Root Mean Squared Error
- R^2 — Coefficient of Determination

Example:

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, predictions)

print("MAE:", mae)
```

---

26. Classification Metrics

Common classification metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

Example:

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, predictions)

print("Accuracy:", accuracy)
```

Important

Accuracy is not always sufficient.

For example, if only 1% of transactions are fraudulent, a model that predicts "not fraud" for every transaction can have very high accuracy while being useless for detecting fraud.

Therefore, the evaluation metric should match the business problem.

---

27. Sentiment Analysis

Sentiment analysis attempts to classify text according to sentiment categories.

A simple binary sentiment problem may use:

```
Positive
Negative
```

Example:

```
"I really enjoyed this product."

→ Positive
```

```
"The service was extremely poor."

→ Negative
```

A practical ML approach is:

```
Text
 ↓
Convert text into numerical features
 ↓
Train classification model
 ↓
Prediction
```

One common classical approach is:

```
TF-IDF + Logistic Regression
```

TF-IDF converts text into numerical features based on term importance, and logistic regression can then be used as a classifier.

---

28. Rule-Based Chatbot vs Machine-Learning Chatbot

A rule-based chatbot uses predefined rules.

Example:

```
if message == "hello":
```

```
    response = "Hello!"
```

It does not learn from conversations.

A machine-learning chatbot instead uses a trained model to classify or interpret user input.

For this introductory chapter, we will first build a **rule-based chatbot** because it teaches:

- Input processing
- String normalization
- Conditions
- Intent-like routing
- Functions
- Conversation loops

Later, this can be extended into ML/NLP-based systems.

---

29. 30 Practical Python Programs

The following programs are ordered from beginner to intermediate.

They are intentionally based on practical data-processing and machine-learning scenarios.

Program 1 — Check Python and scikit-learn Installation

```
import sys
import sklearn

print("Python version:", sys.version)
print("Scikit-learn version:", sklearn.__version__)
```

Program 2 — Create a Small Student Dataset

```
students = [
    {"name": "Amit", "hours": 2, "marks": 45},
    {"name": "Riya", "hours": 4, "marks": 58},
    {"name": "Rahul", "hours": 5, "marks": 65},
    {"name": "Sneha", "hours": 7, "marks": 78},
]

for student in students:
    print(
        student["name"],
        "studied for",
        student["hours"],
        "hours and scored",
        student["marks"]
    )
```

Program 3 — Prepare Features and Target

```
import numpy as np

hours = np.array([2, 3, 4, 5, 6, 7, 8])
marks = np.array([42, 48, 55, 63, 70, 76, 84])

X = hours.reshape(-1, 1)
y = marks

print("Features:")
print(X)
```

```
print("\nTarget:")
print(y)
```

Important Concept

`X` contains input features.

`y` contains the target.

Machine-learning libraries generally expect feature data in a 2-dimensional structure:

```
number of samples × number of features
```

Program 4 — Student Marks Prediction Using Linear Regression

```
import numpy as np
from sklearn.linear_model import LinearRegression

hours = np.array([1, 2, 3, 4, 5, 6, 7, 8]).reshape(-1, 1)
marks = np.array([35, 42, 48, 55, 63, 70, 76, 84])

model = LinearRegression()

model.fit(hours, marks)

new_hours = np.array([[6.5]])

prediction = model.predict(new_hours)

print("Predicted marks:", prediction[0])
```

Program 5 — Student Marks Prediction with Train/Test Split

This is a more realistic workflow than training and testing on exactly the same data.

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

hours = np.array([
    1, 2, 3, 4, 5, 6, 7, 8,
    2, 4, 6, 7
]).reshape(-1, 1)

marks = np.array([
    35, 42, 48, 55, 63, 70, 76, 84,
    40, 57, 68, 78
])

X_train, X_test, y_train, y_test = train_test_split(
    hours,
    marks,
    test_size=0.25,
    random_state=42
)

model = LinearRegression()

model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

```
mae = mean_absolute_error(y_test, predictions)

print("Actual marks:", y_test)
print("Predicted marks:", predictions)
print("MAE:", mae)
```

Program 6 — Predict Employee Salary from Experience

```
import numpy as np

from sklearn.linear_model import LinearRegression

experience = np.array([
    1, 2, 3, 4, 5, 6, 7, 8
]).reshape(-1, 1)

salary = np.array([
    25000, 30000, 36000, 42000,
    50000, 58000, 67000, 76000
])

model = LinearRegression()

model.fit(experience, salary)

years = float(input("Enter years of experience: "))

prediction = model.predict([[years]])

print("Estimated salary:", round(prediction[0], 2))
```

This is an educational model. Real salary prediction would require substantially more relevant variables and appropriate data.

Program 7 — Predict Monthly Electricity Consumption

```
import numpy as np

from sklearn.linear_model import LinearRegression

temperature = np.array([
    20, 22, 24, 26, 28, 30, 32, 34
]).reshape(-1, 1)

units = np.array([
    180, 190, 205, 220, 245, 275, 310, 345
])

model = LinearRegression()

model.fit(temperature, units)

today_temperature = float(
    input("Enter today's average temperature: ")
)

prediction = model.predict([[today_temperature]])

print(
    "Predicted electricity consumption:",
    round(prediction[0], 2),
    "units"
)
```

Program 8 — Evaluate a Regression Model with MAE and R^2

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

X = np.array([
    [1000],
    [1200],
    [1400],
    [1600],
    [1800],
    [2000],
    [2200],
    [2400]
])

y = np.array([
    35,
    42,
    48,
    55,
    62,
    70,
    77,
    85
])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
```

```
    random_state=42
)

model = LinearRegression()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

mae = mean_absolute_error(y_test, predictions)
r2 = r2_score(y_test, predictions)

print("Actual:", y_test)
print("Predicted:", predictions)
print("MAE:", mae)
print("R²:", r2)
```

Program 9 — Predict Delivery Time

```
import numpy as np

from sklearn.linear_model import LinearRegression

distance = np.array([
    2, 4, 6, 8, 10, 12, 15
]).reshape(-1, 1)

delivery_time = np.array([
    15, 22, 30, 38, 48, 57, 70
])

model = LinearRegression()

model.fit(distance, delivery_time)
```

```
customer_distance = float(
    input("Enter delivery distance in km: ")
)

prediction = model.predict([[customer_distance]])

print(
    "Estimated delivery time:",
    round(prediction[0], 1),
    "minutes"
)
```

Program 10 — Customer Churn Classification

Here we predict:

```
0 → Not Churned
1 → Churned
```

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X = np.array([
    [2, 5],
    [24, 1],
    [8, 4],
    [36, 0],
    [6, 5],
```

```
    [30, 1],
    [10, 4],
    [40, 0],
    [5, 6],
    [28, 1]
])

# Columns:
# Months with company, support tickets

y = np.array([
    1, 0, 1, 0, 1,
    0, 1, 0, 1, 0
])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

model = LogisticRegression()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("Predictions:", predictions)
print("Actual:", y_test)

print(
    "Accuracy:",
```

```
    accuracy_score(y_test, predictions)
)
```

Program 11 — Predict Whether a Customer Will Churn

```
import numpy as np

from sklearn.linear_model import LogisticRegression

X = np.array([
    [2, 5],
    [24, 1],
    [8, 4],
    [36, 0],
    [6, 5],
    [30, 1],
    [10, 4],
    [40, 0],
    [5, 6],
    [28, 1]
])

y = np.array([
    1, 0, 1, 0, 1,
    0, 1, 0, 1, 0
])

model = LogisticRegression()

model.fit(X, y)

months = float(
```

```
    input("Months with company: ")
)

tickets = int(
    input("Number of support tickets: ")
)

prediction = model.predict([[months, tickets]])

probability = model.predict_proba(
    [[months, tickets]]
)[0][1]

if prediction[0] == 1:
    print("Prediction: Customer may churn")
else:
    print("Prediction: Customer may remain")

print(
    "Estimated churn probability:",
    round(probability * 100, 2),
    "%"
)
```

Program 12 — Classify Support Tickets

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

data = pd.DataFrame({
```

```
    "message": [
        "I cannot login to my account",
        "My password reset is not working",
        "The payment was deducted twice",
        "I was charged two times",
        "My application crashes when I open it",
        "The website shows an error",
        "I forgot my password",
        "There is an incorrect payment",
        "The app is crashing",
        "I cannot access my account"
    ],
    "category": [
        "account",
        "account",
        "billing",
        "billing",
        "technical",
        "technical",
        "account",
        "billing",
        "technical",
        "account"
    ]
})

X_train, X_test, y_train, y_test = train_test_split(
    data["message"],
    data["category"],
    test_size=0.3,
    random_state=42,
    stratify=data["category"]
)

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
```

```
    ("classifier", LogisticRegression(max_iter=1000))
])

model.fit(X_train, y_train)

message = input("Enter customer message: ")

prediction = model.predict([message])

print("Predicted category:", prediction[0])
```

Program 13 — Simple Positive/Negative Sentiment Classification

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

texts = [
    "I love this product",
    "The product is excellent",
    "Very happy with the service",
    "The service was fantastic",
    "I really enjoyed the experience",
    "This is an amazing product",
    "I hate this product",
    "The product is terrible",
    "Very disappointed with the service",
    "The service was horrible",
    "I had a bad experience",
    "This product is useless"
]
```

```
labels = [
    "positive",
    "positive",
    "positive",
    "positive",
    "positive",
    "positive",
    "negative",
    "negative",
    "negative",
    "negative",
    "negative",
    "negative"
]

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression())
])

model.fit(texts, labels)

message = input("Enter a review: ")

prediction = model.predict([message])

print("Sentiment:", prediction[0])
```

Program 14 — Sentiment Analysis with Confidence

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.pipeline import Pipeline

texts = [
    "excellent service",
    "great product",
    "very happy",
    "amazing experience",
    "good quality",
    "poor service",
    "terrible product",
    "very disappointed",
    "bad experience",
    "horrible quality"
]

labels = [
    "positive",
    "positive",
    "positive",
    "positive",
    "positive",
    "negative",
    "negative",
    "negative",
    "negative",
    "negative"
]

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression())
])

model.fit(texts, labels)

message = input("Enter customer review: ")
```

```
prediction = model.predict([message])[0]

probabilities = model.predict_proba([message])[0]

classes = model.classes_

print("Sentiment:", prediction)

for label, probability in zip(classes, probabilities):
    print(
        label,
        ":",
        round(probability * 100, 2),
        "%"
    )
```

Important

The probability produced by a classifier should not automatically be interpreted as a perfectly calibrated real-world probability.

It is a model output and its quality depends on the training data, model, preprocessing, and calibration.

Program 15 — Spam Message Classification

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

messages = [
    "Congratulations you won a free prize",
    "Claim your free reward now",
```

```
    "You have won a lottery",
    "Click this link to claim your prize",
    "Meeting is scheduled for tomorrow",
    "Please send the project report",
    "Your appointment is confirmed",
    "Can you review this document",
    "The server maintenance is scheduled",
    "Please attend the meeting"
]

labels = [
    "spam",
    "spam",
    "spam",
    "spam",
    "normal",
    "normal",
    "normal",
    "normal",
    "normal",
    "normal"
]

X_train, X_test, y_train, y_test = train_test_split(
    messages,
    labels,
    test_size=0.3,
    random_state=42,
    stratify=labels
)

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression())
])
```

```
model.fit(X_train, y_train)

message = input("Enter message: ")

prediction = model.predict([message])

print("Classification:", prediction[0])
```

Program 16 — Calculate Classification Accuracy

```
from sklearn.metrics import accuracy_score

actual = [
    "spam",
    "normal",
    "spam",
    "normal",
    "normal"
]

predicted = [
    "spam",
    "normal",
    "normal",
    "normal",
    "normal"
]

accuracy = accuracy_score(
    actual,
    predicted
)
```

```
print("Accuracy:", accuracy)
print("Accuracy percentage:", accuracy * 100)
```

Program 17 — Confusion Matrix for Fraud Detection

```
from sklearn.metrics import confusion_matrix

actual = [
    1, 0, 1, 1, 0,
    0, 1, 0, 0, 1
]

predicted = [
    1, 0, 1, 0, 0,
    0, 1, 1, 0, 1
]

matrix = confusion_matrix(
    actual,
    predicted
)

print(matrix)
```

Interpretation for binary classification:

```
True Positive
False Positive
True Negative
False Negative
```

These values are important in applications such as fraud detection, medical screening, security detection, and spam filtering.

Program 18 — Generate a Classification Report

```
from sklearn.metrics import classification_report

actual = [
    "fraud",
    "normal",
    "fraud",
    "normal",
    "normal",
    "fraud"
]

predicted = [
    "fraud",
    "normal",
    "normal",
    "normal",
    "fraud",
    "fraud"
]

print(
    classification_report(
        actual,
        predicted,
        zero_division=0
    )
)
```

This reports metrics such as:

```
precision
recall
f1-score
support
```

Program 19 — Customer Segmentation with K-Means

Although this chapter focuses mainly on regression and classification, clustering is an important ML concept.

```python
import numpy as np

from sklearn.cluster import KMeans

customers = np.array([
    [25, 1500],
    [28, 1700],
    [30, 1800],
    [45, 5000],
    [48, 5200],
    [50, 5500],
    [22, 1000],
    [24, 1200]
])

model = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
```

```
clusters = model.fit_predict(customers)

for customer, cluster in zip(customers, clusters):
    print(
        "Customer:",
        customer,
        "Cluster:",
        cluster
    )
```

This introduces **unsupervised learning**, where the training data does not contain a target label.

Program 20 — Decision Tree for Loan Approval

```
import numpy as np

from sklearn.tree import DecisionTreeClassifier

X = np.array([
    [25, 30000],
    [35, 60000],
    [45, 90000],
    [29, 40000],
    [50, 100000],
    [23, 25000],
    [40, 75000],
    [31, 50000]
])

# 1 = approved
# 0 = not approved
```

```
y = np.array([
    0, 1, 1, 0,
    1, 0, 1, 1
])

model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

model.fit(X, y)

age = float(input("Applicant age: "))
income = float(input("Annual income: "))

prediction = model.predict([
    [age, income]
])

if prediction[0] == 1:
    print("Model classification: Approved")
else:
    print("Model classification: Not approved")
```

Important Real-World Note

A real loan-approval system should not be implemented by blindly copying this example. Real credit decisions require appropriate data, validation, governance, legal/regulatory controls, fairness analysis, and domain-specific rules.

Program 21 — Predict Product Demand

```
import numpy as np

from sklearn.linear_model import LinearRegression
```

```
advertising = np.array([
    10, 20, 30, 40, 50, 60, 70, 80
]).reshape(-1, 1)

sales = np.array([
    100, 125, 150, 180,
    205, 230, 260, 290
])

model = LinearRegression()

model.fit(advertising, sales)

budget = float(
    input("Advertising budget: ")
)

prediction = model.predict([[budget]])

print(
    "Estimated sales:",
    round(prediction[0], 2)
)
```

Program 22 — Predict House Price Using Multiple Features

Linear regression can use multiple input features.

```
import numpy as np

from sklearn.linear_model import LinearRegression
```

```
X = np.array([
    [1000, 2],
    [1200, 2],
    [1500, 3],
    [1800, 3],
    [2000, 4],
    [2200, 4],
    [2500, 4],
    [2800, 5]
])

# Columns:
# house size in sq ft
# number of bedrooms

y = np.array([
    45, 52, 65, 75,
    85, 95, 110, 125
])

model = LinearRegression()

model.fit(X, y)

size = float(
    input("House size in sq ft: ")
)

bedrooms = int(
    input("Number of bedrooms: ")
)

prediction = model.predict([
    [size, bedrooms]
])
```

```
print(
    "Predicted price:",
    round(prediction[0], 2),
    "lakh"
)
```

Program 23 — Handle Missing Values Before ML

Real datasets often contain missing values.

```
import pandas as pd

from sklearn.impute import SimpleImputer

data = pd.DataFrame({
    "age": [22, 25, None, 30, 35],
    "salary": [25000, None, 35000, 42000, 50000]
})

print("Original data:")
print(data)

imputer = SimpleImputer(
    strategy="median"
)

data[["age", "salary"]] = imputer.fit_transform(
    data[["age", "salary"]]
)

print("\nAfter handling missing values:")
print(data)
```

Program 24 — Standardize Features

Feature scaling can be important for many ML algorithms.

```
import pandas as pd

from sklearn.preprocessing import StandardScaler

data = pd.DataFrame({
    "age": [20, 25, 30, 35, 40],
    "salary": [25000, 40000, 55000, 70000, 90000]
})

scaler = StandardScaler()

scaled_data = scaler.fit_transform(data)

print(scaled_data)
```

Important

Feature scaling is not universally required for every algorithm.

For example, tree-based models generally do not require feature scaling in the same way that distance-based or gradient-based methods often do.

Program 25 — Build a Complete ML Pipeline

A pipeline helps keep preprocessing and model training together.

```
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
```

```python
from sklearn.metrics import accuracy_score

X = np.array([
    [20, 25000],
    [25, 30000],
    [30, 45000],
    [35, 55000],
    [40, 70000],
    [45, 85000],
    [50, 100000],
    [55, 120000]
])

y = np.array([
    0, 0, 0, 1,
    1, 1, 1, 1
])

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])

pipeline.fit(X_train, y_train)

predictions = pipeline.predict(X_test)

print(
```

```
    "Accuracy:",
    accuracy_score(y_test, predictions)
)
```

Pipelines are useful because they help apply preprocessing consistently as part of the model workflow.

Program 26 — Save a Trained Model

A trained model can be serialized for later use.

```
import joblib
import numpy as np

from sklearn.linear_model import LinearRegression

X = np.array([
    [1],
    [2],
    [3],
    [4],
    [5]
])

y = np.array([
    35,
    45,
    55,
    65,
    75
])

model = LinearRegression()

model.fit(X, y)
```

```
joblib.dump(
    model,
    "student_marks_model.joblib"
)

print("Model saved successfully.")
```

Install `joblib` if necessary:

```
pip install joblib
```

Program 27 — Load a Trained Model

```
import joblib

model = joblib.load(
    "student_marks_model.joblib"
)

hours = float(
    input("Hours studied: ")
)

prediction = model.predict([
    [hours]
])

print(
    "Predicted marks:",
    round(prediction[0], 2)
)
```

Security Note

Do not blindly load serialized model files from untrusted sources. Serialization mechanisms can have security implications.

Program 28 — Build a Rule-Based Customer Support Chatbot

This introduces chatbot architecture without pretending that a rule-based system is an ML model.

```
def chatbot(message):
    message = message.lower().strip()

    if "hello" in message or "hi" in message:
        return "Hello! How can I help you?"

    if "password" in message:
        return "You can reset your password from the account
settings."

    if "refund" in message:
        return "Please provide your order number for refund a
ssistance."

    if "delivery" in message:
        return "Please provide your order number to check del
ivery status."

    if "thank" in message:
        return "You're welcome!"

    return (
        "I am not sure I understood that. "
        "Please contact customer support."
```

```
    )


while True:
    message = input("You: ")

    if message.lower().strip() in {
        "exit",
        "quit",
        "bye"
    }:
        print("Bot: Goodbye!")
        break

    print("Bot:", chatbot(message))
```

Program 29 — ML-Based Customer Support Intent Classifier

This extends the chatbot concept using machine learning.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

training_messages = [
    "hello",
    "hi there",
    "good morning",
    "I forgot my password",
    "I cannot login",
    "reset my password",
    "where is my order",
    "track my delivery",
```

```
    "when will my order arrive",
    "I want a refund",
    "how can I get my money back",
    "I need to return my order"
]

intents = [
    "greeting",
    "greeting",
    "greeting",
    "password",
    "password",
    "password",
    "delivery",
    "delivery",
    "delivery",
    "refund",
    "refund",
    "refund"
]

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("classifier", LogisticRegression(max_iter=1000))
])

model.fit(
    training_messages,
    intents
)

responses = {
    "greeting":
        "Hello! How can I help you?",

    "password":
```

```
        "You can reset your password from account settings.",

    "delivery":
        "Please provide your order number to check delivery s
tatus.",

    "refund":
        "Please provide your order number to start the refund
process."
}

while True:

    message = input("You: ").strip()

    if message.lower() in {"exit", "quit", "bye"}:
        print("Bot: Goodbye!")
        break

    intent = model.predict([message])[0]

    print("Bot:", responses[intent])
```

This is a small **intent-classification chatbot**.

It is still very different from a modern generative AI assistant or large language model.

Program 30 — Complete Mini Project: Customer Review Analyzer

This final program combines:

- Text processing
- TF-IDF

- Classification
- Train/test split
- Evaluation
- Prediction

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report
)

reviews = [
    "Excellent product and amazing quality",
    "Very happy with my purchase",
    "The product works perfectly",
    "Great quality and fast delivery",
    "I really liked this product",
    "Amazing service and good support",
    "Very satisfied with the product",
    "The product is useful and reliable",

    "Terrible product and poor quality",
    "Very disappointed with my purchase",
    "The product stopped working",
    "Bad quality and slow delivery",
    "I really disliked this product",
    "Poor service and bad support",
    "Very unhappy with the product",
    "The product is unreliable"
]
```

```
sentiments = [
    "positive",
    "positive",
    "positive",
    "positive",
    "positive",
    "positive",
    "positive",
    "positive",

    "negative",
    "negative",
    "negative",
    "negative",
    "negative",
    "negative",
    "negative",
    "negative"
]

X_train, X_test, y_train, y_test = train_test_split(
    reviews,
    sentiments,
    test_size=0.25,
    random_state=42,
    stratify=sentiments
)

model = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            lowercase=True,
            stop_words="english"
        )
    ),
```

```
    (
        "classifier",
        LogisticRegression(
            max_iter=1000
        )
    )
])

model.fit(
    X_train,
    y_train
)

predictions = model.predict(
    X_test
)

print("Model Evaluation")
print("----------------")

print(
    "Accuracy:",
    accuracy_score(
        y_test,
        predictions
    )
)

print(
    "\nClassification Report:"
)

print(
    classification_report(
        y_test,
        predictions,
```

```
        zero_division=0
    )
)

while True:

    review = input(
        "\nEnter customer review "
        "(or type exit): "
    )

    if review.lower() == "exit":
        break

    prediction = model.predict([
        review
    ])[0]

    print(
        "Predicted sentiment:",
        prediction
    )
```

30. What the Student Should Understand After These Programs

By the end of these exercises, the learner should understand this basic ML pattern:

```
Business Problem
      ↓
Collect Data
      ↓
Identify Features
      ↓
Identify Target
```

```
      ↓
Prepare Data
      ↓
Split Data
      ↓
Choose Model
      ↓
Train
      ↓
Predict
      ↓
Evaluate
      ↓
Improve
```

31. Regression vs Classification — Quick Reference

Problem	Output	Type
Predict student marks	Number	Regression
Predict house price	Number	Regression
Predict sales	Number	Regression
Predict delivery time	Number	Regression
Detect spam	Category	Classification
Detect fraud	Category	Classification
Sentiment analysis	Category	Classification
Customer churn	Category	Classification
Support-ticket category	Category	Classification

32. Important Beginner Mistakes to Avoid

Mistake 1 — Training and testing on the same data

Incorrect approach:

```
model.fit(X, y)

predictions = model.predict(X)
```

This does not provide a proper estimate of performance on unseen data.

Prefer a suitable evaluation strategy such as a train/test split or cross-validation.

Mistake 2 — Thinking high accuracy always means a good model

For an imbalanced dataset, accuracy can be misleading.

Always understand:

```
Class distribution
Precision
Recall
F1-score
Confusion matrix
```

when appropriate.

Mistake 3 — Treating correlation as causation

Suppose:

```
Ice cream sales ↑
Drowning incidents ↑
```

This does not mean:

```
Ice cream causes drowning.
```

Both could be influenced by another variable such as hot weather.

Machine learning can identify predictive relationships without proving causation.

Mistake 4 — Using the wrong algorithm for the problem

Example:

```
Predict salary → Regression
Predict spam → Classification
```

The problem should be defined before selecting the algorithm.

Mistake 5 — Data leakage

Data leakage happens when information that should not be available to the model during training is allowed to influence the training process.

This can produce unrealistically good evaluation results.

A common example is fitting preprocessing using information from the test set before evaluation.

Use proper train/test separation and pipelines where appropriate.

Mistake 6 — Assuming the model understands the data

A model does not understand data in the human sense.

For example:

```
"Excellent service"
```

must first be represented numerically before many traditional ML algorithms can process it.

With TF-IDF:

```
Text
 ↓
```

```
Numerical representation
 ↓
ML algorithm
 ↓
Prediction
```

---

33. Suggested Learning Sequence

For a beginner, teach the chapter in this order:

```
Step 1
AI
 ↓
Step 2
ML
 ↓
Step 3
DL
 ↓
Step 4
Python in AI/ML
 ↓
Step 5
Dataset
 ↓
Step 6
Features and Target
 ↓
Step 7
Training and Testing
 ↓
Step 8
Supervised Learning
 ↓
Step 9
Regression
 ↓
Step 10
Linear Regression
 ↓
Step 11
Classification
 ↓
Step 12
Logistic Regression
 ↓
Step 13
```

```
Model Evaluation
 ↓
Step 14
Text Classification
 ↓
Step 15
Sentiment Analysis
 ↓
Step 16
Rule-Based Chatbot
 ↓
Step 17
ML-Based Intent Chatbot
```

---

34. Recommended Practical Progression

Beginner

Start with:

```
Program 1
Program 2
Program 3
Program 4
Program 5
Program 6
Program 7
```

Focus on:

- Python
- NumPy
- Dataset
- Features
- Target
- `fit()`
- `predict()`

Beginner → Intermediate

Then teach:

```
Program 8
Program 10
Program 11
Program 16
Program 17
Program 18
```

Focus on:

- Train/test split
- Classification
- Logistic regression
- Accuracy
- Confusion matrix
- Precision
- Recall
- F1-score

Intermediate

Then:

```
Program 12
Program 13
Program 14
Program 15
Program 23
Program 24
Program 25
```

Focus on:

- Text features

- TF-IDF
- Pipelines
- Missing values
- Scaling
- Classification workflows

Project Level

Finally:

```
Program 28
Program 29
Program 30
```

Focus on:

- Chatbot logic
- Intent classification
- NLP
- ML pipeline
- Model evaluation
- Building a small end-to-end application

---

35. Important Concept Before Moving to Advanced AI

After completing this chapter, students should **not** jump directly into large language models or complex deep-learning architectures.

A better progression is:

```
Python
   ↓
NumPy
```

```
   ↓
Pandas
   ↓
Data Cleaning
   ↓
Data Visualization
   ↓
Machine Learning Fundamentals
   ↓
Regression
   ↓
Classification
   ↓
Model Evaluation
   ↓
Feature Engineering
   ↓
NLP Fundamentals
   ↓
Deep Learning
   ↓
Neural Networks
   ↓
Computer Vision / NLP
   ↓
Generative AI
   ↓
LLMs
   ↓
Prompt Engineering
   ↓
RAG / AI Applications
```

This gives the learner a much stronger foundation than simply learning how to call an AI API.

---

36. Key Takeaways

Remember these core ideas:

```
AI
= Broad field of building intelligent systems

ML
= Systems learn patterns from data
```

```
DL
= Machine learning using multi-layer neural networks

Regression
= Predict a continuous numerical value

Classification
= Predict a category

Feature
= Input used by the model

Target
= Value the model tries to predict

Training
= Learning from training data

Testing
= Evaluating on unseen data

fit()
= Train the model

predict()
= Generate predictions

Evaluation
= Measure how well the model performs
```

The most important lesson is:

> **Machine learning is not simply about choosing an algorithm. The quality of the problem definition, data, preprocessing, evaluation methodology, and deployment process all affect the usefulness of the resulting system.**